

Scalability & Future Plan

Date	30 June 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Single-threaded Flask development server	Cannot handle production-level concurrent traffic beyond local testing.	High
2	Static ML model (no retraining pipeline)	Model predictions may degrade over time as climate patterns shift.	Medium
3	No real-time weather API integration	Users must manually enter meteorological data instead of auto-fetching it.	Low

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Flask development server on localhost (handles ~53 Req/sec).	Deploy using Gunicorn (multi-worker WSGI) behind Nginx reverse proxy for production-grade concurrency.
2	Data Storage	Single PostgreSQL instance with SQLAlchemy ORM.	Migrate to managed cloud PostgreSQL (e.g., AWS RDS or Supabase) with read replicas for high availability.
3	Performance	Flask-Caching with SimpleCache (in-memory, single process).	Upgrade to Redis-backed distributed caching for multi-server deployments.
4	Security	Werkzeug password	Implement JWT-based token

		hashing with Flask session cookies.	authentication and HTTPS enforcement via SSL certificates.
--	--	-------------------------------------	--

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Integrate a live weather API (e.g., OpenWeatherMap) for automatic data fetching.	3 Months	Eliminates manual data entry; enables real-time continuous monitoring.
Phase 3	Build an automated model retraining pipeline using new incoming prediction data.	6 Months	Keeps the XGBoost model accurate as climate patterns evolve over time.
Phase 4	Deploy to cloud (Render/AWS) with CI/CD pipeline and add mobile-responsive PWA support.	9 Months	Makes the application publicly accessible with automatic deployment on every code push.